

Concurrent Programming - Second Work Assignment

Introduction

This document contains the requirements for the second work assignment of the Concurrent Programming (*Programação Concorrente*) course. The solution to this work assignment must be delivered until the end of 2025-05-04, via the creation (and push) of the **0.2.0** Git tag. For each requirement:

- Include technical documentation with the solution design, e.g. as a KDoc to the complete class (e.g. [example](#)).
- Include automated tests, including both basic functional tests and *stress tests*.

Requirement 1

Implement the *thread pool executor synchronizer*, in which each submitted *continuation* is executed on one of the worker threads that the synchronizer creates and manages for this purpose. The public interface of this synchronizer is the following one:

The `ThreadPoolExecutor` class

```
class ThreadPoolExecutor(  
    private val maxThreadPoolSize: Int,  
    private val keepAliveTime: Duration,  
) {  
    @Throws(RejectedExecutionException::class)  
    fun execute(continuation: Continuation<Unit>): Unit { ... }  
  
    fun shutdown(): Unit { ... }  
  
    @Throws(InterruptedException::class)  
    fun awaitTermination(timeout: Duration): Boolean { ... }  
}
```

The maximum number of worker threads (`maxThreadPoolSize`) and the maximum time that a worker thread can remain idle before terminating (`keepAliveTime`) are passed as `ThreadPoolExecutor` constructor arguments. The management of the worker threads by the synchronizer must follow these criteria:

- If the total number of worker threads is less than the specified maximum limit and no worker thread is available, a new worker thread is created whenever a continuation is submitted for execution.
- Worker threads should terminate after the amount of time specified in `keepAliveTime` has passed

without them being assigned a continuation to execute.

- The number of worker threads in the pool at any given moment depends on its activity and can vary between zero and `maxThreadPoolSize`.

Threads that want to execute continuations through the thread pool executor call the `execute` method. This method returns immediately, without waiting for the continuation to be executed or even submitted to a worker thread.

Calling the `shutdown` method puts the executor in *shutdown* mode and returns immediately. In this mode, all subsequent calls to the `execute` method must throw a `RejectedExecutionException`. However, all continuations submitted before the call to `shutdown` must be processed normally.

The `awaitTermination` method allows any calling thread to synchronize with the completion of the executor's shutdown process. That is, it waits until all accepted callables are executed and all active worker threads have terminated. It may finish:

- normally, returning true if the shutdown of the executor completes;
- normally, returning false if the timeout specified as an argument expires before shutdown completes;
- exceptionally, by throwing `InterruptedException` if the calling thread is interrupted while waiting.

Requirement 2

Add the following function to the `ThreadPoolExecutor` executor:

The `ThreadPoolExecutor` class and the suspend `awaitTermination`.

```
class ThreadPoolExecutor(  
    private val maxThreadPoolSize: Int,  
    private val keepAliveTime: Duration,  
) {  
  
    suspend fun awaitTermination(): Unit { ... }  
}
```

The `awaitTermination` function suspends the calling coroutine until the shutdown of the executor is completed. The `awaitTermination` does not need to support coroutine cancellation.

Requirement 3

Consider the following definitions

```
data class PeriodicMessage(  
    val delayBetweenStrings: Duration,  
    val nOfRepetitions: Int,
```

```

        val strings: List<String>,
    ) {
        init {
            require(nOfRepetitions > 0)
        }
    }

    fun show(vararg periodicMessages: PeriodicMessage) {...}

```

Implement the `show` function that concurrently shows all periodic messages provided as arguments. The `show` function returns when all periodic messages are completely shown. Use `suspend` functions so that showing each periodic message is internally accomplished by the following suspend function, where `sleep` is an internal auxiliary function that needs to be implemented.

```

suspend fun showPeriodicMessage(msg: PeriodicMessage) {
    repeat(msg.nOfRepetitions) {
        msg.strings.forEach { string ->
            println(string)
            sleep(msg.delayBetweenStrings)
        }
    }
}

```

The implementation of this requirement must not create any threads, directly or indirectly (e.g. via an executor). Namely, all `println` calls should occur in the thread that called `show`. Do not use any functionality from the `kotlinx.coroutines` package.